

Personal Project — Development & DevOps

Recette API

Design, development and deployment of a secure REST API
with a full CI/CD pipeline and a web interface

Author	Manoach HOSSOU DODO
Backend repository	https://github.com/hdmanoach/recette-api
Frontend repository	https://github.com/hdmanoach/recette-frontend
API demo	https://recette-api-l2r6.onrender.com/docs
Interface demo	https://recette-frontend-five.vercel.app

Built in 2026

Document generated with L^AT_EX

Contents

1	Introduction and context	2
1.1	Origin of the project	2
1.2	Objectives	2
1.3	Overview of the result	2
2	Overall architecture	3
2.1	Overview	3
2.2	Communication flow	3
2.3	Splitting the two repositories	3
3	Backend — FastAPI API	4
3.1	Technology choices	4
3.2	Project structure	4
3.3	Data modeling	4
3.3.1	Recipes table	5
3.3.2	Validation with Pydantic	5
3.4	Authentication and security	5
3.4.1	Registration and password hashing	5
3.4.2	Login and JWT token generation	5
3.4.3	Route protection	6
3.4.4	Ownership control	6
3.4.5	Brute-force attack protection	6
3.5	Logging	6
3.6	API endpoints	7
3.7	Configuration management	7
3.8	Automatic SQLite / PostgreSQL switch	7
3.9	Health check endpoint	8
4	Testing, quality and continuous integration	9
4.1	Automated tests	9
4.2	Static code analysis	9
4.3	GitHub Actions pipeline	9
4.4	Containerization with Docker	10
5	Frontend — Next.js interface	11
5.1	Technology choices	11
5.2	Main pages	11
5.3	Client-side authentication handling	11
5.4	Communication with the API	11

6	Production deployment	13
6.1	Backend on Render, database on Supabase	13
6.2	Frontend on Vercel	13
6.3	CORS configuration	13
7	Challenges encountered and solutions	15
8	Skills applied and acquired	16
8.1	Backend development	16
8.2	Frontend development	16
8.3	Software quality and DevOps	16
9	Possible improvements	17
10	Conclusion	18

Chapter 1

Introduction and context

1.1 Origin of the project

This project started from a GitHub repository initially named `github-actions-demo`, created to learn how to set up a continuous integration and continuous deployment (CI/CD) pipeline with GitHub Actions. Rather than starting from an abstract example, the choice was made to build a real end-to-end application: a recipe management API, paired with a web interface, so that every concept learned (automated testing, linting, containerization, deployment) would apply to a concrete, coherent use case.

The project was developed progressively and pedagogically: each feature was added step by step, tested, fixed, and integrated into the quality pipeline before moving on to the next one.

1.2 Objectives

- Design a complete REST API with FastAPI (Python).
- Set up a persistent relational database.
- Secure the API with a JWT-based authentication system.
- Automate code quality (tests, linting) via GitHub Actions.
- Containerize the application with Docker.
- Deploy the API and a web interface to production, with continuous deployment.
- Document the whole approach and architecture.

1.3 Overview of the result

The final project consists of two separate applications, deployed independently and communicating over HTTP :

- a **backend** built with FastAPI, exposing a REST API, deployed on **Render**, connected to a PostgreSQL database managed by **Supabase** ;
- a **frontend** built with Next.js, consuming this API, deployed on **Vercel**.

Concrete result: a working, publicly accessible application, with authentication, per-user content management, and a continuous integration chain that automatically validates every code change before it reaches production.

Chapter 2

Overall architecture

2.1 Overview

The architecture follows a classic client-server model, with a clear separation between the frontend (user interface) and the backend (business logic and data).

Frontend	Backend	Database
Next.js (React, TypeScript) Deployed on Vercel HTTP requests (fetch)	FastAPI (Python) Deployed on Render REST API + JWT	PostgreSQL (Supabase, production) SQLite (development) Managed via SQLAlchemy (ORM)

2.2 Communication flow

1. The user loads the Next.js interface in their browser.
2. The frontend sends HTTP requests (`fetch`) to the API, at the address defined by the `NEXT_PUBLIC_API_URL` environment variable.
3. The FastAPI backend processes the request: data validation (Pydantic), authentication check if required (JWT), database query via SQLAlchemy.
4. The JSON response is sent back to the frontend, which updates the display.

To allow this communication between different origins (`localhost:3000` in development, `vercel.app` in production, towards `onrender.com`), FastAPI's **CORS** middleware was explicitly configured.

2.3 Splitting the two repositories

The choice was made to use **two separate Git repositories** rather than a single monorepo :

- **recette-api**: the backend, with its own CI/CD pipeline and its own deployment (Render).
- **recette-frontend**: the frontend, with its own deployment (Vercel).

This separation reflects a common practice in the industry: each application has its own lifecycle, its own tests, and can be deployed independently of the other.

Chapter 3

Backend — FastAPI API

3.1 Technology choices

Technology	Role
FastAPI	Python web framework, automatic OpenAPI/Swagger documentation generation
SQLAlchemy	ORM (Object-Relational Mapping), database abstraction
Pydantic	Validation of data entering and leaving the API
PostgreSQL (Supabase) / SQLite	Relational database (production / development)
python-jose	Generation and verification of JWT tokens
passlib + bcrypt	Secure password hashing
pytest	Automated tests
ruff	Static analysis and code style (linting)
Docker	Application containerization

3.2 Project structure

The code was progressively reorganized, moving from a single monolithic `main.py` file to a modular architecture based on FastAPI **routers** :

```
recette-api/  
|-- main.py           # Entry point: app, CORS, middleware, routers  
|-- database.py       # Database connection  
|-- models.py         # Pydantic models (validation) and SQLAlchemy (tables)  
|-- security.py       # JWT, password hashing, brute-force protection  
|-- middleware.py     # Logging of every request  
|-- routers/  
|   |-- auth.py       # /register, /login  
|   |-- recipes.py    # Recipe CRUD  
|   '-- logs.py       # /logs  
|-- test_main.py      # Automated tests (pytest)  
|-- requirements.txt  
|-- pyproject.toml    # Ruff configuration  
|-- Dockerfile  
'-- .github/workflows/ci.yml # CI/CD pipeline
```

This breakdown clearly separates responsibilities: security (`security.py`), observability (`middleware.py`), and each business domain in its own router.

3.3 Data modeling

Two families of models are used on the backend side, a distinction that is essential to understand :

- **Pydantic** models (`Recipe`, `UserCreate`, `UserLogin`, `UserOut`) validate the JSON data exchanged with API clients ;
- **SQLAlchemy** models (`RecipeDB`, `UserDB`, `LogEntry`) describe the actual database tables.

3.3.1 Recipes table

```
class RecipeDB(Base):
    __tablename__ = "recipes"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    title = Column(String, nullable=False)
    ingredients = Column(JSON, nullable=False)
    instructions = Column(JSON, nullable=False)
    prep_time = Column(Integer, nullable=False)
    servings = Column(Integer, nullable=False)
    owner_id = Column(Integer, ForeignKey("users.id"))
```

The `owner_id` field establishes a foreign key to the users table: each recipe belongs to a single owner.

3.3.2 Validation with Pydantic

The `Recipe` class enforces business rules before the data even reaches the database :

```
class Recipe(BaseModel):
    id: int | None = None
    title: str = Field(..., min_length=1)
    ingredients: list[str] = Field(..., min_length=1)
    instructions: list[str] = Field(..., min_length=1)
    prep_time: int = Field(..., gt=0)
    servings: int = Field(..., ge=1, le=20)
```

Any request that does not satisfy these constraints (empty title, negative preparation time, etc.) is automatically rejected by FastAPI with a 422 error code, without any business logic having to be written manually.

3.4 Authentication and security

3.4.1 Registration and password hashing

Passwords are never stored in plain text. The `passlib` library, using the `bcrypt` algorithm, computes an irreversible hash :

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

3.4.2 Login and JWT token generation

Upon login, a signed **JWT** (JSON Web Token) is generated and returned to the client. This token contains the user's identity (`sub`) and an expiration date, without requiring any server-side session storage :

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

3.4.3 Route protection

A FastAPI dependency (`get_current_user`) decodes and verifies the token presented in the `Authorization: Bearer <token>` header, then retrieves the corresponding user from the database. It is injected into the endpoints that require authentication :

```
@router.post("/recipes")
def create_recipe(
    recipe: Recipe,
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user),
):
    ...
```

3.4.4 Ownership control

Beyond simple authentication, an **authorization** check ensures that a user can only modify or delete their own recipes :

```
if recipe.owner_id != current_user.id:
    raise HTTPException(
        status_code=403,
        detail="You are not allowed to delete this recipe",
    )
```

3.4.5 Brute-force attack protection

An in-memory counter records failed login attempts per username. After five failures within fifteen minutes, the account is temporarily locked out (HTTP 429) :

```
MAX_LOGIN_ATTEMPTS = 5
LOCKOUT_DURATION_MINUTES = 15

def is_locked_out(username: str) -> bool:
    attempts = failed_login_attempts.get(username, [])
    now = datetime.now(timezone.utc)
    recent = [a for a in attempts
               if now - a < timedelta(minutes=LOCKOUT_DURATION_MINUTES)]
    failed_login_attempts[username] = recent
    return len(recent) >= MAX_LOGIN_ATTEMPTS
```

3.5 Logging

A **middleware** intercepts every HTTP request received by the API and records it in the database: method, path, status code, IP address, and username (extracted from the JWT token when present). These logs can be viewed via the protected `GET /logs` endpoint, with pagination and descending date sorting.

3.6 API endpoints

Method	Route	Description
GET	/	Welcome message
GET	/health	Checks API status and database connection
POST	/register	Create a user account
POST	/login	Log in, obtain a JWT token
GET	/recipes	List recipes (filters, pagination)
GET	/recipes/{id}	Retrieve a recipe
POST	/recipes	Create a recipe (<i>authenticated</i>)
POST	/recipes/bulk	Create several recipes (<i>authenticated</i>)
PUT	/recipes/{id}	Update a recipe (<i>owner only</i>)
DELETE	/recipes/{id}	Delete a recipe (<i>owner only</i>)
GET	/logs	View the request log (<i>authenticated</i>)

The GET /recipes endpoint accepts several optional query parameters: `skip` and `limit` for pagination, `title` and `ingredient` for a partial, case-insensitive text search, and `max_prep_time` to filter by maximum preparation time.

3.7 Configuration management

Sensitive information (JWT secret key, database URL, allowed CORS origins) is never hardcoded in the source code. It is loaded from environment variables via `python-dotenv`, with a local `.env` file excluded from the Git repository :

```
load_dotenv()
SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "30"))
```

In production (Render) and in the CI pipeline (GitHub Actions), these same variables are injected via each platform's own secrets system.

3.8 Automatic SQLite / PostgreSQL switch

To allow simple local development with no external dependency, while guaranteeing real persistence in production, `database.py` dynamically selects the database engine :

```
DATABASE_URL = os.getenv("DATABASE_URL")

if DATABASE_URL:
    if DATABASE_URL.startswith("postgres://"):
        DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
        engine = create_engine(DATABASE_URL)
    else:
        os.makedirs("data", exist_ok=True)
        engine = create_engine(
            "sqlite:///./data/recettes.db",
            connect_args={"check_same_thread": False},
        )
```

If the `DATABASE_URL` variable is defined (production case), PostgreSQL is used. Otherwise, the application falls back to SQLite, stored locally in a file.

Networking note: Render's free tier does not support IPv6, while Supabase's *direct* connection URI resolves over IPv6. It is therefore essential to use Supabase's **Connection Pooling (Session)** URI, on port 6543 rather than the direct port 5432, for the connection to succeed from Render.

3.9 Health check endpoint

A `GET /health` endpoint was added for two reasons :

- to check at any time that the API is responding and that the database connection is working (useful for quick diagnostics) ;
- to keep the **Supabase** database active, since free Supabase projects can be paused after a prolonged period of inactivity.

A free external service (cron-job.org) automatically calls this endpoint every 72 hours, which is enough to keep the Supabase project active without manual intervention.

Chapter 4

Testing, quality and continuous integration

4.1 Automated tests

Tests, written with `pytest` and FastAPI's `TestClient`, use an **in-memory SQLite** database, fully isolated from the development or production database. An important technical issue had to be resolved: by default, each in-memory SQLite connection creates a separate database. The solution is to force a single, shared connection via `StaticPool` :

```
engine = create_engine(
    "sqlite:///memory:",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
```

The application's `get_db` dependency is replaced (`dependency_overrides`) with a version pointing to this test database. A `pytest` fixture creates the tables before each test and drops them afterwards, guaranteeing full isolation between tests.

4.2 Static code analysis

The `ruff` tool is run systematically to check style and detect common errors (unused imports, badly sorted imports, misused variables). An explicit exception was configured in `pyproject.toml` for rule `B008`, which incorrectly flags the standard, recommended use of `Depends(...)` in FastAPI function signatures :

```
[tool.ruff.lint]
ignore = ["B008"]
```

4.3 GitHub Actions pipeline

The CI/CD pipeline is automatically triggered on every **push** or pull request to the main branch. It runs two jobs in parallel :

- **test**: installs dependencies, checks style (`ruff`), runs the tests (`pytest`) ;
- **docker-build**: builds the Docker image, to guarantee that the application remains containerizable with every change.

```

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest
    env:
      SECRET_KEY: ${ secrets.SECRET_KEY }

  docker-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t recette-api .

```

The secret key required for authentication tests is injected from the GitHub repository's secrets, never appearing in plain text in the code or in execution logs.

4.4 Containerization with Docker

The Docker image is built while optimizing layer caching: dependencies are installed before the rest of the source code is copied, so that a change to the code alone does not force a full reinstallation of Python packages.

```

FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}

```

Local data persistence (outside production) is provided by a **Docker volume**, mapping the container's `data/` folder to a folder on the host system :

```
docker run -p 8000:8000 -v $(pwd)/data:/app/data recette-api
```

Chapter 5

Frontend — Next.js interface

5.1 Technology choices

The frontend was built with **Next.js** (App Router), **TypeScript** and **Tailwind CSS**, in line with current standards of the React ecosystem.

5.2 Main pages

Route	Role
/	List of recipes (public access), edit actions when logged in
/login	Login form, JWT token storage
/register	Registration form
/recipes/new	Create a recipe (authenticated)
/recipes/[id]/edit	Edit an existing recipe (dynamic route)

5.3 Client-side authentication handling

After a successful login, the JWT token returned by the API is stored in the browser's `localStorage`, then reused in the `Authorization` header of every request that requires authentication :

```
const token = localStorage.getItem("token");

await fetch(`${process.env.NEXT_PUBLIC_API_URL}/recipes`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify(recipeData),
});
```

The interface's login state (conditional display of the "Edit" / "Delete" buttons, switch between "Log in" and "Log out") is derived from the presence of this token.

5.4 Communication with the API

The API address is injected via a public environment variable :

```
NEXT_PUBLIC_API_URL=https://recette-api-12r6.onrender.com
```

The `NEXT_PUBLIC_` prefix is required by Next.js for a variable to be accessible on the browser side, and not only on the server side.

Chapter 6

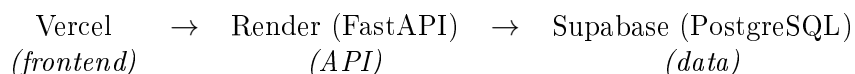
Production deployment

6.1 Backend on Render, database on Supabase

The backend is deployed on **Render**, built directly from the repository's `Dockerfile`. Every **push** to the main branch triggers an automatic redeployment.

Data persistence is provided by a **PostgreSQL database managed by Supabase**, chosen after a first attempt using Render's own free PostgreSQL offering: that one expires after a few weeks on the free plan, whereas Supabase offers a permanent free tier (subject to regular activity, hence the `/health` endpoint paired with an external cron job described in the previous chapter).

The production architecture can be summarized as follows :



to which **cron-job.org** is added, which queries `/health` every 72 hours to keep the Supabase project active.

Production environment variables are configured directly in the Render dashboard, separately from the source code :

Variable	Value
DATABASE_URL	Supabase connection pooling (session) URI, port 6543
SECRET_KEY	JWT signing secret key
ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	30
CORS_ORIGINS	Allowed origins (local and deployed frontend)

6.2 Frontend on Vercel

The frontend is deployed on **Vercel**, the platform built by the creators of Next.js, with automatic framework detection and continuous deployment on every **push**.

6.3 CORS configuration

To allow requests from the deployed frontend to the API, the allowed origins are defined dynamically from an environment variable, rather than hardcoded :

```
cors_origins_raw = os.getenv(
    "CORS_ORIGINS", "http://localhost:3000"
)
origins = [o.strip() for o in cors_origins_raw.split(",")]
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=origins,  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```


Chapter 7

Challenges encountered and solutions

This section retraces the main technical obstacles encountered during development, and how they were resolved — a part deliberately kept in this document because it records real learning, useful for any future project with a similar structure.

Problem	Cause and solution
<code>passlib</code> / <code>bcrypt</code> incompatibility	Recent versions of <code>bcrypt</code> removed an internal attribute expected by <code>passlib</code> . Solution: pin the version to <code>bcrypt==4.0.1</code> .
422 error on the Swagger authorization form	<code>OAuth2PasswordBearer</code> expects form data, while <code>/login</code> received JSON. Solution: replaced with <code>HTTPBearer</code> , better suited to a token pasted directly.
Tables not found in tests (<code>no such table</code>)	Each in-memory SQLite connection creates a separate database. Solution: use of <code>StaticPool</code> to force a single, shared connection.
Misleading CORS error in production	A server error (500) occurring before CORS headers are added can appear as a CORS error in the browser, masking the real cause. Solution: systematically checking server logs (Render) to identify the actual error.
<code>integer out of range</code> on PostgreSQL	An identifier generated client-side with <code>Date.now()</code> (a millisecond timestamp) exceeded the capacity of a standard PostgreSQL integer, without causing issues on SQLite (more permissive). Solution: let the database generate identifiers automatically (<code>autoincrement</code>), removing manual generation on the client side.
Lost PostgreSQL connection locally	The internal URL provided by Render can only be resolved from within Render's private network. Solution: only define <code>DATABASE_URL</code> in production environment variables, never locally.
Expiration of the free Render database	Render's free PostgreSQL offering is time-limited (a few weeks). Solution: migration to Supabase, whose free tier is permanent subject to regular project activity.
Render → Supabase connection failure	Supabase's <i>direct</i> connection URI resolves over IPv6, which is not supported by Render's free plan. Solution: use of the connection pooling URI (port 6543), IPv4-compatible.

Chapter 8

Skills applied and acquired

8.1 Backend development

- Designing a REST API with FastAPI (routing, validation, dependencies).
- Relational data modeling with SQLAlchemy (ORM).
- JWT-based authentication and secure password hashing.
- Authorization management (resource ownership).

8.2 Frontend development

- Building an interface with Next.js (App Router) and TypeScript.
- React state management (`useState`, `useEffect`) and dynamic routes.
- Consuming an authenticated REST API from the client.

8.3 Software quality and DevOps

- Writing isolated automated tests (pytest).
- Static code analysis (ruff).
- Containerization with Docker (image, volumes, caching best practices).
- Building a CI/CD pipeline with GitHub Actions (parallel jobs, secrets).
- Continuous deployment on cloud platforms (Render, Vercel).
- Configuration management via environment variables and secrets.

Chapter 9

Possible improvements

- **User roles:** introduce an administrator role with extended rights (access to all users' logs, moderation).
- **Token refresh:** add a refresh token mechanism to extend a session without a full new login.
- **Recipe categorization:** add a category field (starter, main course, dessert) with a dedicated filter.
- **Image upload:** attach a photo to each recipe, with external storage (S3 or equivalent).
- **Frontend tests:** add automated tests on the interface side (Jest, Testing Library, or Playwright for end-to-end tests).
- **Enhanced observability:** export logs to a dedicated service rather than storing them in the application database.
- **httpOnly cookies:** replace token storage in `localStorage` with secure cookies, more robust against XSS attacks.

Chapter 10

Conclusion

This project, which started as a simple demo repository for GitHub Actions, made it possible to build a complete, working application covering the whole modern software development chain: designing a secure API, data persistence, user interface, and automation of testing and deployment.

Beyond the technical outcome, the progressive approach adopted — one feature at a time, systematically tested before moving on to the next — helped build a solid understanding of each building block of the architecture, rather than a superficial assembly of technologies. The difficulties encountered along the way, documented in Chapter 6, form a useful reference base for any future projects with a similar structure.
